Expert Systems With Applications 102 (2018) 179–192

Contents lists available at ScienceDirect

Expert Systems With Applications

journal homepage: www.elsevier.com/locate/eswa

Multi-objective particle swarm optimization algorithm based on objective space division for the unequal-area facility layout problem

Jingfa Liu[a,b], Huiyun Zhang[a,*], Kun He[c], Shengyi Jiang[b]

[a] School of Computer & Software, Nanjing University of Information Science & Technology, Nanjing 210044, China
[b] School of Information Science and Technology, Guangdong University of Foreign Studies, Guangdong 51006, China
[c] School of Computer Science and Technology, Huazhong University of Science and Technology, Wuhan 430074, China

ARTICLE INFO

Article history:
Received 26 August 2017
Revised 25 February 2018
Accepted 26 February 2018
Available online 27 February 2018

Keywords:
Multi-objective optimization
Facility layout problem
Particle swarm optimization
Preference
Group decision-making

ABSTRACT

The facility layout problem (FLP) is the problem of placing facilities in a certain shop floor so that facilities do not overlap each other and are satisfied with some given objectives. Considering practical situations, this study focuses on the multi-objective unequal-area facility layout problem (UA-FLP), where the facilities have unequal-areas and fixed shapes and are placed orthogonally in the continuous shop floor. The objectives of the problem aim to optimize the material handling cost, the total adjacency value and the utilization ratio of the shop floor. The chief difficulties of this version of the FLP lie in the satisfaction of non-overlapping constraint between any two different facilities and the optimization of multiple objectives in the huge solution space. In this paper, we put forward a heuristic configuration mutation operation and subsequent local search based on the gradient method to satisfy the non-overlapping constraint, and the multi-objective particle swarm optimization (MOPSO) algorithm, which has recently proven its high effectiveness and robustness in solving multi-objective problems, to obtain a set of Pareto-optimal solutions of the problem. The novelty of the paper lies in the use of an objective space division method in the MOPSO which governs the neighborhood topology and the local best swarm used to assess the global fitness of a solution and choose the global leader particle. The proposed algorithm is tested on three sets of different UA-FLPs from the literature with the size of the problem up to 62 facilities. The numerical results show that the proposed method is effective in solving the multi-objective UA-FLP.
© 2018 Elsevier Ltd. All rights reserved.

1. Introduction

The facility layout problem (FLP) concerns the most efficient physical arrangement of a set of interacting facilities with known dimensions within the manufacturing system in order to satisfy one or more objectives. In manufacturing organizations, facilities occupy a large proportion of the assets and are of great importance (Nordin, Zainuddin, Salim, & Ponnusamy, 2009). Tompkins, White, Bozer, and Tanchoco (2010) found that approximately 20%–50% of operating cost are usually allocated to facility planning and material handling, and an effective layout design can reduce that cost by 10%–20%.

Since the FLP belongs to the NP-hard problem, in recent years many researchers have developed a lot of approaches to find (sub-) optimal solutions to the FLP. Recent developments are mostly based on iterative meta-heuristic approaches such as simulated annealing (SA) algorithm (Chwif, Barretto, & Moscato, 1998), genetic algorithms (GAs) (Aiello, Enea, & Galante, 2006; Al-Hakim, 2000; Diego, Santamarina, Alcaide, & Cloquell, 2009; Islier, 1998), Tabu search (TS) method (Mckendall & Hakobyan, 2010), particle swarm optimization (PSO) algorithm (Jolai, Tavakkoli-Moghaddam, & Taghipour, 2012), ant colony optimization (ACO) algorithm (Komarudin & Wong, 2010, Wong & Komarudin, 2010; Kulturel-Konak & Konak, 2010), Wang–landau (WL) sampling algorithm (Liu, Wang, He, & Xue, 2017) and so on. Classical approaches to the FLP focus on the relative locations of equal-area facilities on a shop floor. Then, the FLP can be modeled as the quadratic assignment problem (QAP) of allocating equal-area facilities to a number of definite centroid locations. However, equal-area facilities are very poor assumption in most applications and real-world competitive markets. When layout problems have varying area facilities, the FLP should be treated as unequal-area facility layout problem (UA-FLP).

In order to handle the UA-FLP, different approaches have been proposed. Wu and Appleton (2002) designed a GA with tree structure to represent a layout and considered the material flows and free spaces between different facilities in the evaluation func-

* Corresponding author.
E-mail addresses: jfliu@nuist.edu.cn (J. Liu), zhanghuiyun1992@foxmail.com (H. Zhang), brooklet60@hust.edu.cn (K. He).

https://doi.org/10.1016/j.eswa.2018.02.035
0957-4174/© 2018 Elsevier Ltd. All rights reserved.

tion. García-Hernández, Pérez-Ortiz, Araúzo-Azofra, Salas-Morera, and Hervás-Martínez (2014) proposed a hybrid system by incorporating human expert knowledge into the FLP. A subset of facility designs was generated via the GA and then evaluated by human expert's knowledge. Kulturel-Konak and Konak (2015) presented a large-scale hybrid SA algorithm to solve four different UA-FLPs on the continuous plane. Asl and Wong (2017) developed a modified PSO algorithm which applied two local search methods and the facility swapping method to improve the quality of solutions and prevent local optima. Kang and Chae (2017) presented a harmony search (HS)-based heuristic algorithm to solve UA-FLPs, and a slicing tree representation (STR) was used as a form of layout structure. Chang and Ku (2013) developed a heuristic method by combining the STR and a quadratic constraint program (QCP) model with the HS to solve the UA-FLP. A hybrid algorithm based on the variable neighborhood search and ACO was proposed by Guan and Lin (2016) to solve the single row UA-FLP. Gonçalves and Resende (2015) established a linear programming model and proposed a biased random-key GA for the UA-FLP. The quality of their approach was validated by the improvement of the best known solutions for 19 of the 28 extensively studied benchmark datasets. Palomo-Romero, Salas-Morera, and García-Hernández (2017) proposed an island model genetic algorithm (IMGA) to solve the UA-FLP, and the parallel evolution of several populations was used to maintain the population diversity and to obtain a wider sampling of the search space. Ulutas and Kulturel-Konak (2012) proposed a clonal selection algorithm with a new encoding and a novel procedure to copy with dummy departments to solve the UA-FLP with the flexible bay structure. The algorithm showed consistent performance for the 25 test problem cases studied. Vitayasak, Pongcharoen, and Hicks (2017) focused on solving facility layout problems using GAs and the backtracking search algorithm (BSA). The BSA was a population-based iterative evolutionary algorithm designed to achieve global optimization. Paes, Pessoa, and Vidal (2017) introduced two algorithmic approaches to address the UA-FLP: a basic GA and a GA combined with a decomposition strategy via partial solution deconstructions and reconstructions. Their algorithm had been tested on classical benchmark instances from the literature, and returned high-quality solutions.

Although the approaches mentioned above have made great progress in the UA-FLP over the past decades, most of the relevant researches focused only on single objective, either quantitative or qualitative feature of the layout. In reality, different facility layout problems (FLPs) involve different issues (Drira, Pierreval, & Hajri-Gabouj, 2007), depending on the characteristics of manufacturing shop floor (e.g., facility shape, aisle and elevator considerations, number of floors). Meanwhile, several conflicting objectives (such as material handling cost, adjacency requirements, utilization ratio of the shop floor, safety, and so on) need to be considered simultaneously in practical FLPs. Therefore, a layout that is optimal with respect to one criterion might perform poorly when a different criterion is considered. Before reaching any conclusion, both quantitative and qualitative objectives must be satisfied. Considering these demands, heuristic approaches and intelligent algorithms based on Pareto optimization have received a lot of attentions in the past few decades due to their ability to generate feasible solutions in the least possible computational time for the UA-FLPs with different objectives and constraints. Aiello et al. (2006) suggested a multi-objective model that considered material handling cost, adjacency requirements, distance requirements and aspect ratio requirements at the same time. The Pareto-optimal solutions were determined by the multi-objective GA and the selection of the optimal solution was based on the multi-criteria decision-making procedure Electre. Zhou, Ye, Cao, and Ye (2006) developed a multi-objective GA with local search to solve a multi-objective nonlinear mixed-integer FLP model considering aisles, material handling cost and adjacent requirement between resources. Aiello, Scalia, and Enea (2012) came up with a multi-objective GA based on slicing structure by means of a Pareto-based evolutionary approach, and the problem considered four objectives (material handling cost, aspect ratio, closeness and distance requests) simultaneously. Li, Yan, Xie, and Huang (2012) built a multi-objective model for optimizing the material handling cost, adjacent relationship and utilization rate of area. They divided the shop floor into several rows and applied a non-dominated sorting genetic algorithm II (NSGA-II) with the elite strategy to solve it. Zhang, Lu, Zhang, and Fang (2013) put forward a differential cellular multi-objective GA. Through introduction of the differential evolution strategy into the canonical cellular multi-objective GA, the algorithm integrated features of cellular algorithm's good diversity, differential evolution strategy's good convergence and wide covering area. Gómez, Fernández, Garciá, and Garciá (2003) formulated a multi-objective model considering the material handling flow, closeness and distance ratios requirement in the production process. They used Muther's SLP method proposed by Muther (1973) to resolve layout problems in shop floors where there are aisles. Khilwani, Shankar, and Tiwari (2008) gave a multi-person, multi-criteria and multi-preference scenario for FLPs. They developed a psycho-clonal evolutionary algorithm with an ordered weighted averaging operator for the aggregation of the decision maker's information. Saraswat, Venkatadri, and Castillo (2015) designed a multi-objective meta-heuristic framework that integrated sequence pair representation, ε-accurate model and multi-objective SA algorithm to solve the FLP with three objectives (flow-distance, average work-in-process, and the number of required material handling devices). Jolai et al. (2012) implemented a multi-objective particle swarm optimization (MOPSO) to find near optimal solutions for unequal sized dynamic facility layout problem with pichup/drop-off locations. Aiello, Scalia, and Enea (2013) employed a multi-objective GA to determine Pareto-optimal solutions of four objective functions which referred to material handling cost, adjacent relationship, distance requirement and aspect ratio of facilities, and the subsequent selection of the optimal solution was carried out by means of the multi-criteria decision-making procedure Electre. Although the approaches mentioned above solve the multi-objective UA-FLP effectively, their efficiency is still low, calling for further improvement.

In this paper, a modified multi-objective particle swarm optimization (MOPSO) algorithm is used to solve the multi-objective UA-FLP. This is because the MOPSO is not only easy to implement where there are few parameters to adjust, but it has good convergence speed and is also one of the successfully established solution approaches to multi-objective optimization problems. The objectives of the work include optimizing the material handling cost, the adjacency value and the utilization ratio of the shop floor simultaneously. The main contributions of this paper are embodied in two aspects: (1) the use of an objective space division method in the MOPSO which governs the neighborhood topology and the local best swarm used to assess the global fitness of a solution and choose the global leader particle; (2) the use of a heuristic configuration mutation operation and subsequent local search based on the gradient method used to satisfy the non-overlapping constraint. Furthermore, in application, three groups of instances taken from the literature are performed. The first two groups of instances are modeled from practical applications while the third group of instances provides benchmark datasets for analyzing the performance of the proposed algorithm, as well as demonstrating the applicability of the proposed algorithm in solving various types of FLPs.

The remainder of the paper is organized as follows. Section 2 presents the mathematical model of the UA-FLP.

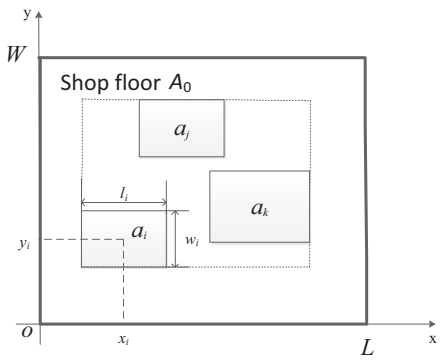


Fig. 1. A schematic diagram of facility layout in a shop floor.

Section 3 describes the improved MOPSO algorithm implemented in this study for the UA-FLP. Section 4 presents numerical experiments and makes a comparison. Finally, conclusions of the paper are provided in Section 5.

2. Problem statement and formulation

Based on the engineering of the placement of factory machinery and equipment, this paper takes the static layout of facilities in a shop floor as the research background. Assume that there are N facilities, named a_1, a_2, ..., a_N, which have unequal areas and interactions between them. The objective is to find an efficient arrangement of these facilities within the given shop floor A_0, and to optimize the following three objectives: the material handling cost, the adjacency value between facilities, and the utilization ratio of the shop floor. We give the following assumptions:

(i) All facilities are rectangular and their shapes and areas are fixed.
(ii) The facilities can only be placed horizontally or vertically.
(iii) The route of handling material is horizontal or vertical, and material flow is from a facility's centroid to another facility's centroid.
(iv) Distance between two different facilities is the Manhattan distance between their centers.

Considering the characteristics of the facility and the convenience of description, we define a coordinates system (Fig. 1) which is used to calculate the centroid of the shop floor and describe the positions of N facilities. The bottom left corner of the shop floor is set as the origin and the x-axis coincides with the length edge of the shop floor and the y-axis coincides with the width edge of the shop floor. Taking into account the impact of external environmental factors and the requirements for real-world applications, we consider that there exists the isolated space between any two facilities and between the shop floor and facilities. The rectangle formed by the dotted line (Fig. 1) is the envelope rectangle which can contain all the facilities.

First, we present some notations for describing the mathematical optimization model of the UA-FLP.

Indexes

i, j indexes for facilities, $i, j = 1, 2, \ldots, N$, where N is the number of facilities

Parameters

C_{ij} cost of a unit material handling per unit distance between facility a_i and facility a_j
F_{ij} frequency of material flow between facility a_i and facility a_j
AV_{ij} adjacency value between facility a_i and facility a_j

Table 1
Range of adjacency factor.

Relationship condition	Adjacency factor af_{ij}
$0 < d_{ij} \le d_{max}/6$	1.0
$d_{max}/6 < d_{ij} \le d_{max}/3$	0.8
$d_{max}/3 < d_{ij} \le d_{max}/2$	0.6
$d_{max}/2 < d_{ij} \le 2d_{max}/3$	0.4
$2d_{max}/3 < d_{ij} \le 5d_{max}/6$	0.2
$5d_{max}/6 < d_{ij} \le d_{max}$	0.0

L length of shop floor
W width of shop floor
l_i length of facility a_i
w_i width of facility a_i
S_i area of facility a_i

Variables

(x_i, y_i) coordinates of centroid of facility a_i
d_{ij} manhattan distance between facility a_i and facility a_j
af_{ij} adjacency factor between facility a_i and facility a_j
Next, a mathematical model of the UA-FLP is presented.

Objective functions:

$$(1)\quad \text{Minimize}: \quad f_1(X) = \sum_{i=1}^{N}\sum_{j=1}^{N}\left(C_{ij} * F_{ij} * d_{ij}\right) = \sum_{i=1}^{N}\sum_{j=1}^{N}\left(C_{ij} * F_{ij} * \left(|x_i - x_j| + |y_i - y_j|\right)\right) \tag{1}$$

$$(2)\quad \text{Maximize}: \quad f_2(X) = \sum_{i=1}^{N}\sum_{j=1}^{N}\left(AV_{ij} * af_{ij}\right) \tag{2}$$

$$(3)\quad \text{Maximize}: \quad f_3(X) = \frac{\sum_{i=1}^{N} S_i}{S_R} \tag{3}$$

Subject to:

$$x_i - 0.5l_i \ge 0, \ \forall\, i \in \{1, 2, \ldots, N\} \tag{4}$$

$$y_i - 0.5w_i \ge 0, \ \forall\, i \in \{1, 2, \ldots, N\} \tag{5}$$

$$x_i + 0.5l_i \le L, \ \forall\, i \in \{1, 2, \ldots, N\} \tag{6}$$

$$y_i + 0.5w_i \le W, \forall\, i \in \{1, 2, \ldots, N\} \tag{7}$$

$$Int(a_i) \cap Int(a_j) = \emptyset, \ i \ne j, \ \forall\, i, j \in \{1, 2, \ldots, N\} \tag{8}$$

Here, $X = (x_1, y_1, \beta_1, x_2, y_2, \beta_2, \ldots, x_N, y_N, \beta_N)$ is a configuration, or a solution, where (x_i, y_i) denotes the coordinates of facility a_i and β_i is the orientation of facility a_i, $1 \le i \le N$. In this paper, we set $\beta_i = 0$ or $\beta_i = \pi/2$. $\beta_i = 0$ indicates that the longer edge of facility a_i is parallel to the x-axis while $\beta_i = \pi/2$ represents that the longer edge of facility a_i is parallel to the y-axis. The first objective function expresses the sum of material handling cost among the facilities to be minimized. The second objective function is modeled by the maximization of the total adjacency value. The adjacency factor af_{ij}, which reflects the adjacency condition between facilities a_i and a_j, depends on the practical distance d_{ij} between facilities a_i and a_j and the maximum possible distance d_{max} between facilities a_i and a_j Table 1). The adjacency value AV_{ij} represents a functional relationship and corresponds to adjacency requirement, which is not always quantifiable and is difficult to be given sometimes. In this paper, the most common quantifiable value for AV_{ij} proposed by Lee, Han, and Roh (2003) is used as

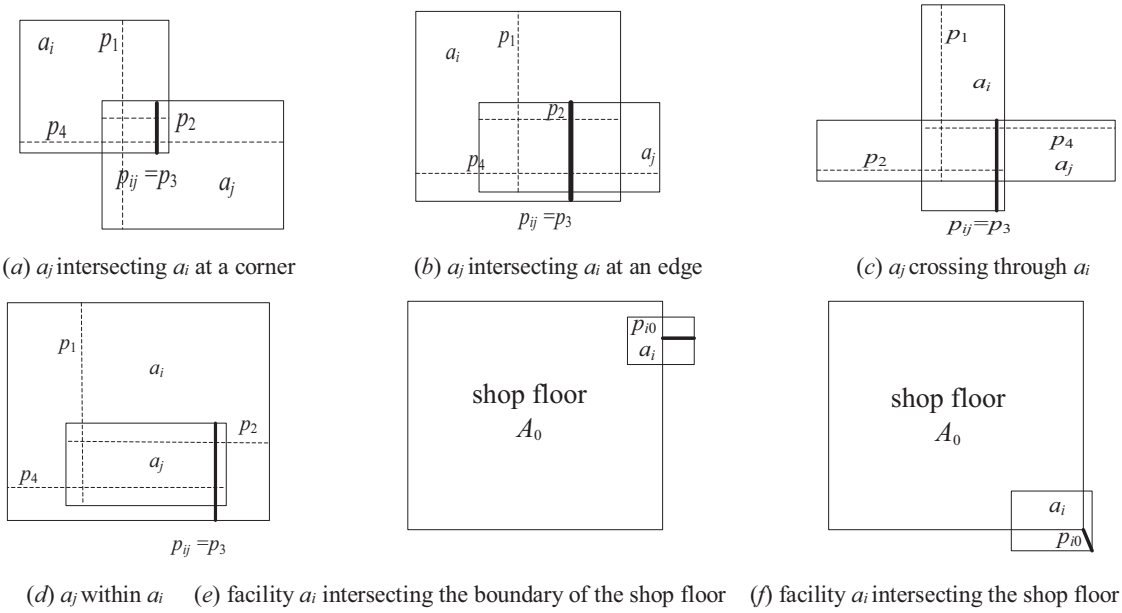


Fig. 2. Embedding depth. (a)–(d) Embedding depth p_{ij} between facilities a_i and a_j. (e)–(f) Embedding depth p_{i0} between facility a_i and shop floor A_0.

Table 2
Adjacency values corresponding to adjacency requirement.

Adjacency requirement	Adjacency value *AVij*	Relationship
A	5	Extremely desirable
E	4	Very desirable
I	3	Desirable
O	2	Indifferent
U	1	Unimportant
X	0	Undesirable

shown in Table 2. The causes of adjacency requirement may be, for instance, the absence or presence of noise or safety, the logical organization of the manufacturing system and so on. Objective ((3) maximizes the utilization ratio of the shop floor. Since S_i represents the area of facility a_i, the total area of all the facilities is a constant. S_R denotes the area of the envelope rectangle of all facilities within the shop floor, so Eq. (3) also can be expressed as minimizing area S_R of the envelope rectangle.

Constraints (4)–(7) assure that all facilities are placed inside of the shop floor. $Int(a_i)$ represents the interior of facility a_i, and constraint (8) ensures that there is no interference (overlapping) between any two different facilities.

Inspired by the quasi-physical approach (Liu et al., 2015; Liu, Xue, Liu, & Xu, 2009), we suppose that all the facilities and the outside of the shop floor are smooth elastic solids. If there exists the embedding between facilities a_i and a_j or between facility a_i and the outside of the shop floor (Fig. 2), the computation of the embedding depth between them is as follows:

(1) If facilities a_i and a_j overlap each other (Fig. 2(a)–(c)), including facility a_j located in facility a_i (Fig. 2(d)), the embedding depth p_{ij} between them is defined as the minimal displacement distance of facility a_j along the x-axis or the y-axis, ensuring non-overlap between facilities a_i and a_j. Here, $p_{ij} = \min\{p_k | k = 1,2,3,4\}$, where p_1, p_2, p_3, and p_4 represent the minimal displacement distance of facility a_j, which moves out of the facility a_i from the top, the right, the bottom, and the left of facility a_i, respectively.
(2) If facility a_i and the outside of the floor shop A_0 overlap each other along the x-axis (or y-axis) direction (Fig. 2(e)) or facility a_i intersects the outside of the shop floor A_0 at a corner (Fig. 2(f)), the embedding depth p_{i0} is defined as the minimal displacement distance of facility a_i which moves into the shop floor.

On the basis of elastic mechanics, when there exists the interference (embedding) between any two different facilities a_i and a_j or between facility a_i and the outside of the shop floor, the extrusive elastic potential energy between them is proportional to the square of the embedding depth of them. So, the extrusive elastic potential energy between them is given by the following equation:

$$E_{ij} = \mu \cdot p_{ij}^2, \ \ i, j \in \{0, 1, 2, \ldots\ldots, N\}, i \neq j \tag{9}$$

where μ is the elastic coefficient. When $j = 0$ (or $i = 0$), E_{i0} is the extrusive elastic potential energy between facility a_i and the outside of the shop floor. Thus, in the whole layout system, the total extrusive elastic potential energy is given by the following equation:

$$E(X) = \sum_{i=1}^{N-1} \sum_{j=i+1}^{N} E_{ij} + \sum_{i=1}^{N} E_{i0} \tag{10}$$

Obviously, $E(X) \geq 0$. For a shop floor with given size in advance, if there exists a configuration X^* such that $E(X^*) = 0$, constraints (4)–(8) are obviously satisfied. This implies that X^* is a feasible solution of the original problem (1)–(8). This can be obtained by executing local search method, the gradient method, for the unconstrained optimization problem minimize $E(X)$. To find Pareto-optimal solutions to the UA-FLP, we propose a modified multi-objective particle swarm optimization algorithm to optimize objectives (1)–(3).

3. Methodology for the UA-FLP

This proposed approach, called the MOPSO–OSD, is described in the following sections.

3.1. Particle swarm optimization algorithm

Particle swarm optimization (PSO) algorithm, first proposed by Kennedy and Eberhart (1995), is based on swarm intelligence and

evolutionary computation. As a representative of bio-inspired algorithm and meta-heuristic algorithm, PSO can solve continuous optimization problems with single objective or multiple objectives and has good convergence speed. Similar to the other evolutionary computation techniques, PSO is a population-based search algorithm and is initialized with a population of random solutions, called particles. Unlike the other evolutionary algorithms, each particle in PSO is associated with an internal velocity. Particles fly through the search space with velocities and are dynamically adjusted according to their historical behaviors. The usage of historical behaviors consists of three factors, which are the previous velocity, the best position of each particle (called Pbest), and the swarm best position (called Gbest). As particles swarm over the solution space, they communicate with each other, broadcasting the fitness of their Pbest and learning about the Pbests of others. Therefore, all the particles tend to converge to the best solution quickly. Considering the simplicity and good convergence speed of PSO, we choose this algorithm to solve the UA-FLP. In this paper, the position of each particle represents a configuration of the UA-FLP, and formulas describing the velocity and the position of particle i are shown in the following equtions (Engelbrecht, 2007), respectively:

$$v_i^{t+1} = \omega * v_i^t + c_1 * rand_1^t \cdot \left(Pbest_i^t - X_i^t\right) + c_2 * rand_2^t \cdot \left(Gbest^t - X_i^t\right) \tag{11}$$

$$X_i^{t+1} = X_i^t + v_i^{t+1} \tag{12}$$

where t is the index of time, and i is the index of particle. $X_i^t = (x_{i1}^t,\ y_{i1}^t,\ \beta_{i1}^t, x_{i2}^t,\ y_{i2}^t,\ \beta_{i2}^t,\ \ldots,\ x_{iN}^t,\ y_{iN}^t,\ \beta_{iN}^t)$ and v_i^t denote the position and the velocity of particle i at time t, respectively. Pbest_i^t represents the position where particle i has had its best fitness so far at time t, while Gbest^t represents the position of the global best fitness so far by any particle in the population at time t. The inertia coefficient ω is used to balance exploration versus exploitation of the solution space. Values of ω close to one usually encourage exploration of new areas in the solution space, and for small values of ω such as $\omega < 0.4$, the search shifts to the exploitation mode. Parameters c_1 and c_2 are called cognition and social coefficients, respectively. They are very important for facilitating the convergence of PSO. Their values are in the range of 0 to 2. $rand_1^t$ and $rand_2^t$ are two vectors of N uniform random numbers between 0 and 1. The operator $(\cdot)$ in Eq. (11) indicates element-by-element vector multiplication.

3.2. Heuristic mutation operation

After each particle updates its position, there exists probably the embedding among facilities. To make the configurations of particles be feasible, we execute the following heuristic mutation operation.

Suppose that $X_i = (x_{i1}, y_{i1}, \beta_{i1}, x_{i2}, y_{i2}, \beta_{i2}, \ldots, x_{iN}, y_{iN}, \beta_{iN})$ is the current position (configuration) of particle i. The relative extrusive elastic potential energy of facility a_k (k = 1, 2,..., N) is defined as $Q_k = E_k/(l_k{}^* w_k)$, where $E_k = \sum_{j=0, j\neq k}^{N} p_{kj}^2$, and the vacant region is defined as the region where all facilities (except facility a_k) have not occupied in the shop floor at that moment. In order to update configuration X_i, we first select a facility (for instance, facility a_l) which has the maximum relative extrusive elastic potential energy from all the facilities, and then generate randomly 20 points within the vacant region of the shop floor. We temporarily put the center of facility a_l at each point and calculate the corresponding Q_l of facility a_l. Thereafter, we choose the position with the smallest Q_l as the new position of the center of facility a_l. With positions of other facilities unchanged, we obtain an updated configuration X_i^{old} corresponding to particle i.

Algorithm 1 GMRA (X_i^{old}).

1: Set the initial iterative step length $h = 1$, and let $h_{min} = 10^{-4}$, $u = 1.2$, $q = 0$, $\varepsilon = 10^{-20}$.
2: In the current configuration X_i^{old}, calculate the gradient vector of $E(X_i^{old})$ on the x-axis, and y-axis, i.e.

$$\nabla(E(X_i^{old})) = \left(\frac{\partial E(X)}{\partial x_1}, \frac{\partial E(X)}{\partial y_1}, \frac{\partial E(X)}{\partial x_2}, \frac{\partial E(X)}{\partial y_2}, \ldots, \frac{\partial E(X)}{\partial x_N}, \frac{\partial E(X)}{\partial y_N}\right)\Big|_{X=X_i^{old}}$$

Then, a new configuration $X_i^{new} = X_i^{old} - h * \nabla(E(X_i^{old}))$ is gained.
3: If $E(X_i^{new}) \geq 1$, then
3.1: If $E(X_i^{new}) > E(X_i^{old})$, then let $h = h^*0.8$, and $q = 0$.
3.2: If $E(X_i^{new}) \leq E(X_i^{old})$, then let $X_i^{old} = X_i^{new}$, and $q = q + 1$.
3.3: If $q \geq 3$, then let $h = h^*u$, and $q = 0$.
3.4: Let $X_i^{new} = X_i^{old} - h * \nabla(E(X_i^{old}))$.
3.5: If $|E(X_i^{new}) - E(X_i^{old})| > 0.01$, then go to step 3.1.
4: Let $q = 0$.
5: If $E(X_i^{new}) < 1$, then
5.1 Execute steps 3.1 to 3.4.
5.2 If $E(X_i^{new}) \geq \varepsilon$ and $h \geq h_{min}$, then go to step 5.1.
6: Return X_i^{new}.

3.3. Local search

After executing the heuristic mutation operation of particle i, we use the adaptive gradient method (GM) with a retreat strategy and an acceleration strategy to execute the local search so as to search the lower-energy configurations around the updated configuration X_i^{old}. In the adaptive GM (Liu & Li, 2010), if the energy $E(X_i^{new})$ of the newly produced configuration X_i^{new} after an iteration is higher than the energy $E(X_i^{old})$ of the previous configuration X_i^{old}, it implies that the iterative step length h of the GM is too large, and the step length h will reduce by multiplying it by 0.8 in next iteration; otherwise, h will remain unchanged. By this strategy, we can achieve the dynamic adaptation of the step length h. Meanwhile, if the energy $E(X_i^{new})$ of the newly produced configuration X_i^{new} is very close to the energy $E(X_i^{old})$ of the previous configuration X_i^{old} and $E(X_i^{new}) \geq 1$, then configuration X_i^{new} is deemed to have no "prospect". At that time, the search may reach to a local minimum and the GM will end in advance. In this way, the so-called "early-escape" strategy will help the adaptive GM to decrease the time of search.

In addition, in the adaptive GM, we notice that the newly generated configurations are always accepted by default. To improve the accuracy and the stability of the adaptive GM, in this paper a retreat strategy is added to the GM. In the process of executing the GM, if the energy of the newly generated configuration is higher than that of the previous configuration, the newly produced configuration is not accepted and the previous configuration is restored as the current configuration. Then, we continue to execute the adaptive GM.

Moreover, in order to decrease the searching time, an acceleration strategy is added into the adaptive GM. If the energy of the newly generated configuration decreases in three successive iterations, let $h = h^*u$ (u is an acceleration coefficient, and $u > 1$). The search accuracy of the GM is improved by using the retreat strategy while the speed of search is enhanced by the acceleration strategy. Thus, an adaptive gradient method with a retreat strategy and an acceleration strategy (GMRA) is proposed.

Based on the current configuration X_i^{old} of particle i, the details of the GMRA (X_i^{old}) are shown in Algorithm 1.

Table 3
Dimensions of facilities.

Facility	1	2	3	4	5	6	7	8	9	10	11	12
Length (l_i)	1.5	1	1.2	0.8	1.3	0.5	1.2	1.1	0.7	0.9	1.3	1
Width (w_i)	1	0.8	0.7	0.9	1	0.4	1.2	0.8	0.9	0.6	1	1.2

Table 4
Matrix of spacing requirement h_{ij} between facilities a_i and a_j.

Facility	1	2	3	4	5	6	7	8	9	10	11	12
1	0	1.2	1.5	1.5	1	1.4	2	0.9	1	1.2	1.1	0.8
2	1.2	0	1.5	1.7	2	1	1.2	1.5	1.6	0.8	1.3	1.2
3	1.5	1.5	0	1.2	1.4	2	0.9	1.4	1.3	1.2	1.4	1.3
4	1.5	1.7	1.2	0	1.3	1.4	1.2	1.1	1	1	0.9	1.6
5	1	2	1.4	1.3	0	1.5	1.7	1.3	1.2	1	0.8	1.2
6	1.4	1	2	1.4	1.5	0	1.3	1.2	0.8	1	1.5	1.8
7	2	1.2	0.9	1.2	1.7	1.3	0	1.1	0.8	1.5	1.2	1.3
8	0.9	1.5	1.4	1.1	1.3	1.2	1.1	0	1.8	1.2	0.9	1
9	1	1.6	1.3	1	1.2	0.8	0.8	1.8	0	1.2	0.9	1.5
10	1.2	0.8	1.2	1	1	1	1.5	1.2	1.2	0	1.3	1.6
11	1.1	1.3	1.4	0.9	0.8	1.5	1.2	0.9	0.9	1.3	0	1.5
12	0.8	1.2	1.3	1.6	1.2	1.8	1.3	1	1.5	1.6	1.5	0

Table 5
Material flows F_{ij} between facilities a_i and a_j.

Facility	1	2	3	4	5	6	7	8	9	10	11	12
1	0	1	2	2	3	0	0	1	1	1	2	3
2	1	0	2	3	1	3	0	1	2	2	0	2
3	2	2	0	2	1	1	0	3	0	1	2	0
4	2	3	2	0	0	2	1	0	1	3	2	0
5	3	1	1	0	0	1	3	3	0	1	2	1
6	0	3	1	2	1	0	0	2	1	3	0	1
7	0	0	0	1	3	0	0	3	0	2	2	2
8	1	1	3	0	3	2	3	0	1	2	0	1
9	1	2	0	1	0	1	0	1	0	3	0	1
10	1	2	1	3	1	3	2	2	3	0	2	2
11	2	0	2	2	2	0	2	0	0	2	0	3
12	3	2	0	0	1	1	2	1	1	2	3	0

Table 6
Matrix of adjacency requirement between two facilities.

Facility	1	2	3	4	5	6	7	8	9	10	11	12
1	–	U	A	O	I	U	U	O	I	U	X	U
2	U	–	I	U	U	O	I	U	O	U	U	X
3	A	I	–	O	U	U	I	U	O	U	O	U
4	O	U	O	–	U	U	I	U	O	U	O	U
5	I	U	E	U	–	O	U	O	U	I	U	O
6	U	O	U	U	O	–	U	U	O	U	O	U
7	U	I	O	I	U	U	–	U	U	U	U	U
8	O	U	U	U	O	U	U	–	U	O	E	I
9	I	O	A	O	U	O	U	U	–	I	O	U
10	U	U	X	U	I	U	U	O	I	–	O	O
11	X	U	I	O	U	O	U	E	O	O	–	U
12	U	X	I	U	O	U	U	I	U	O	U	–

Table 7
Matrix of a unit material handling cost C_{ij} per unit distance between facilities a_i and a_j.

Facility	1	2	3	4	5	6	7	8	9	10	11	12
1	0	8	6	3	5	2	3	1	2	1	3	5
2	8	0	4	3	2	2	4	7	3	5	1	2
3	6	4	0	4	3	2	1	4	5	2	3	2
4	3	3	4	0	7	2	4	5	6	2	1	4
5	5	2	3	7	0	4	5	2	3	1	8	4
6	2	2	2	2	4	0	3	3	7	4	1	5
7	3	4	1	4	5	3	0	2	7	5	3	6
8	1	7	4	5	2	3	2	0	3	6	2	7
9	2	3	5	6	3	7	7	3	0	2	4	2
10	1	5	2	2	1	4	5	6	2	0	3	1
11	3	1	3	1	8	1	3	2	4	3	0	5
12	5	2	2	4	4	5	6	7	2	1	5	0

Table 8
Comparison of the best computational results by three different algorithms for instance 1.

Objective	NSGA-II (Li et al., 2012)	MOPSO–CD	MOPSO–OSD
MHC (min)	2868.7	2860.65	**2856.54**
TAV (max)	161.2	170.2	**178.0**
UR (max)	19.3%	24.98%	**27.04%**

3.4. Objective space division method

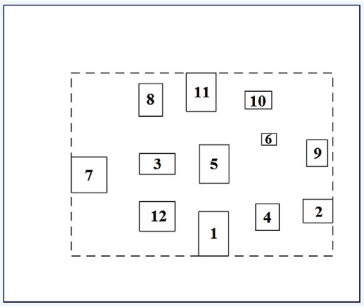
In order to determine the Pbest and Gbest of particles in the MOPSO, we propose an objective space division (OSD) method to evaluate the fitness value for each particle.

Suppose that a multi-objective problem has m objective functions $f_1, f_2,\ldots, f_m$. They constitute a m-dimensional objective space, which can be divided into $k_1 \times k_2 \times \cdots \times k_m$ grids with the same interval, where k_i (i = 1, 2,..., m) is the number of grids of the ith dimensional objective space. It is obvious that the value of k_i will influence the number of particles in each grid. Here, we define $k_i = 100$ (i = 1, 2,..., m). The width d_i of each grid in the ith dimension is computed by the following eqution:

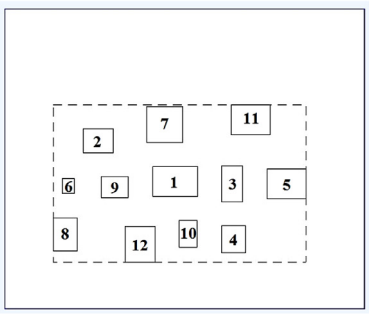
$$d_i = \frac{\max\limits_{X\in D} f_i(X) - \min\limits_{X\in D} f_i(X)}{k_i} \tag{13}$$

Here, X is a configuration, and D consists of all visited configurations.

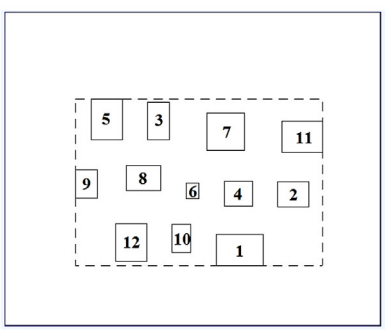
For convenience, we set $F_i^{max} = \max\limits_{X\in D} f_i(X)$, $F_i^{min} = \min\limits_{X\in D} f_i(X)$, and suppose that point $O = (F_1^{min}, F_2^{min}, \ldots, F_m^{min})$ is the origin of the objective space in the current iteration, that is, origin O is the intersection of all the lower bounds in the objective space. For particle i, its corresponding objective vector Z_i is $(f_1^i, f_2^i, \ldots, f_m^i)$, where $f_j^i (1 \le j \le m)$ is the jth objective value of particle i. The distance vector between origin O and point Z_i can be expressed as $(t_1, t_2,\ldots, t_m)$, where $t_j = f_j^i - F_j^{min}$. For the jth objective value of particle i, its relative address in the objective space is $h_i = mod(t_i, d_i) + 1$, where $mod(t_i, d_i)$ denotes the integral part after t_i divided by d_i. The particles, whose all components of the objective vectors have the same address, belong to the same grid. The



(a)The optimal layout based on the preference of the objective 1



(b)The optimal layout based on the preference of the objective 2



(c) The optimal layout based on the preference of the objective 3

Fig. 3. Three Pareto-optimal solutions with different preferences obtained by the MOPSO–OSD algorithm for instance 1.

particles, which locate in the same grid, are called members of this grid.

In the kth iteration, the fitness value of the current position of particle i is defined by the following equation:

$$fit(i,k) = \frac{H(i,k)}{density(i,k)} \tag{14}$$

Here, $H(i, k)$ is the number of the particles dominated by particle i in the kth iteration, and $density(i, k)$ is the number of the particles in the same grid with particle i in the kth iteration. Generally, the higher the fitness value of a particle is, the better its position (configuration) is. The purpose of this definition of the fitness value is to find the Pareto front which is uniformly distributed, well extensible and near-optimal simultaneously.

3.4.1. Finding Pbest

In the beginning of the MOPSO–OSD, we set the initial position (configuration) of each particle as its Pbest. In the later iterations, if the current position of particle i dominates the Pbest, the current position will replace it; otherwise, the Pbest of particle i remains unchanged. If there is no dominance relation between them, we calculate their fitness values by Eq. (14). The position with the higher fitness value will be selected as the Pbest. If their fitness values are equal, we select randomly one of them as the Pbest.

Algorithm 2 MOPSO–OSD.

1: Generate randomly initial position X_i and initial velocity v_i of particle i. Let $t = 1$, $X_i^t = X_i$, and $v_i^t = v_i$. Set the initial position X_i^t of particle i as its $\mathbf{Pbest}_i^t$, $i = 1, 2, \ldots, n$. Find the global best position Gbest^t from positions of all particles.
2: Calculate the objective function values of positions of all particles, and store the non-dominated solutions into the external archive (EA). Let $t = t + 1$.
3: If $t < 10^4$, then go to step 4; otherwise output the EA and exit.
4: Compute velocity v_i^t and position X_i^t of particle i ($i = 1, 2, \ldots, n$) by Eqs. (11) and (12), respectively.
5: Calculate the energy $E(X_i^t)$ of the current position X_i^t of particle i ($i = 1, 2, \ldots, n$) by Eq. (10). If $E(X_i^t) > 0$, it means there is overlapping in the current configuration of particle i. We update the current configuration X_i^t by the heuristic mutation operation, and gain an updated configuration $X_i^{t,\ old}$.
6: Execute the local search GMRA ($X_i^{t,old}$) for particle i, and gain a new configuration $X_i^{t,new}$, $i = 1, 2, \ldots, n$.
7: On the basis of the objective space division method, locate addresses of the new configurations of all the particles in the objective space according to their values of objective functions.
8: Update the current best position $\mathbf{Pbest}_i^t$ of particle i ($i = 1, 2, \ldots, n$).
9: Update the current global best position Gbest^t of all particles, and go to step 2.

3.4.2. Finding Gbest

To find the Gbest, we allocate a non-dominance score to each of grids containing at least one non-dominated solution. The non-dominance score of a grid is equal to one divided by the number of non-dominated solutions that it contains. The non-dominance score of the grid that contains more non-dominated solutions is lower. For all grids which contain at least one non-dominated solution, based on their non-dominance scores, we adopt the roulette-wheel method to select a grid and randomly select a solution from this grid as the Gbest.

3.5. Updating of external archive

For the UA-FLP with multiple objectives, we adopt an external archive (EA) to store Pareto-optimal solutions. After each iteration, we are required to update the contents of the EA by adding all the new non-dominated solutions into the EA if they are not dominated by any of the stored solutions in the archive. It is worth noting that non-dominated solutions are selected from the set of feasible solutions obtained by executing the heuristic mutation operation and subsequent local search. Then all dominated solutions in the EA are removed. Since the size of the archive is limited, whenever it gets full, we apply a secondary criterion for retention: solutions which have higher fitness values (Eq. (14)) are given priority over those which have lower fitness values. Therefrom, we delete the solutions with lower fitness values until the upper limit of the EA is reached.

3.6. Description of the MOPSO–OSD

By integrating the heuristic mutation operation, the local search and the objective space division method into the particle swarm optimization algorithm, a multi-objective particle swarm optimization algorithm based on objective space division (MOPSO–OSD) is put forward for the UA-FLP. The general procedure of the MOPSO–OSD is outlined in Algorithm 2.

Table 9
Statistics of results by the MOPSO–OSD algorithm for instance 1.

Objective	Best objective value	Worst objective value	Average objective value	Standard deviation
MHC(min)	2856.5	3316.84	3073.61	120.5
TAV(max)	178.0	165.6	173.8	3.31
UR(max)	27.04%	20.88%	24.96%	0.02

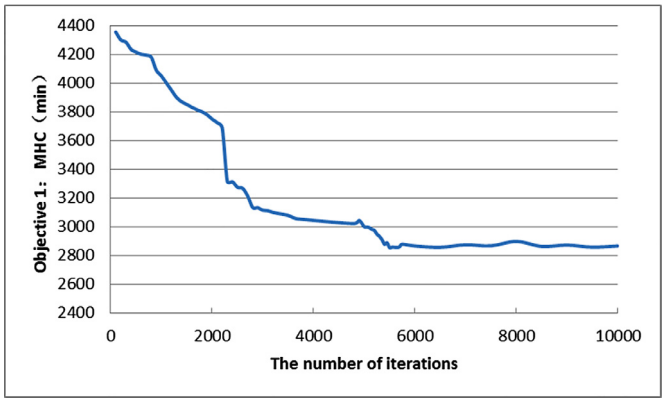


Fig. 4. The evolution process of objective 1.

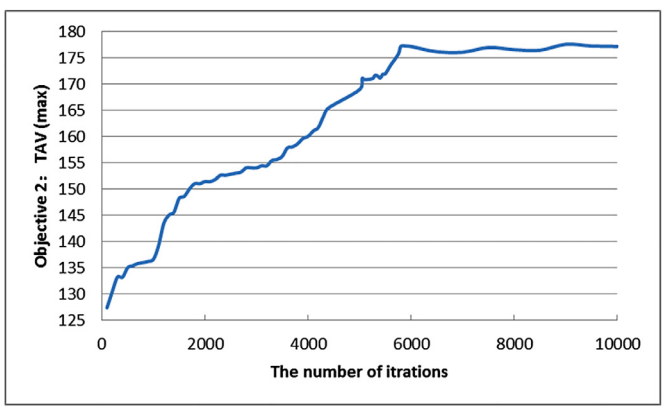


Fig. 5. The evolution process of objective 2.

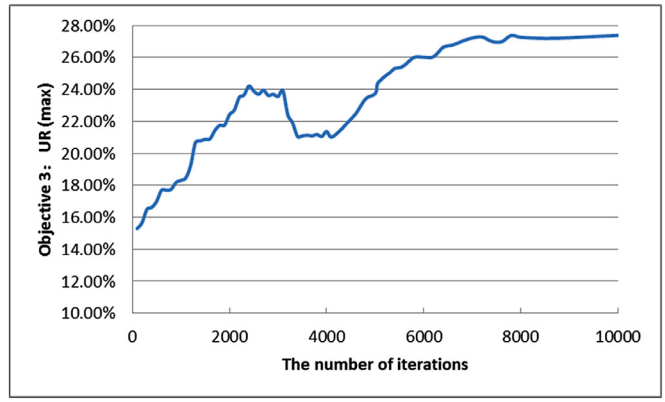


Fig. 6. The evolution process of objective 3.

4. Numerical results and analysis

To evaluate the performance of the MOPSO–OSD algorithm, three groups of instances from the literature are performed. The first two groups of instances are modeled from practical applications while the third group of instances provides benchmark datasets. Three groups of different instances are used to test the performance of the MOPSO–OSD algorithm and demonstrate the applicability of the proposed algorithm in solving various types of FLPs. The number of particles in the algorithm is set at 100 for each of three groups of instances. The algorithm is compiled by Java language, and run on a Notebook PC with Intel Core 2 Duo, 2.60 GHz processor and 2.0 GB of RAM. For each instance, we run the algorithm ten times, independently.

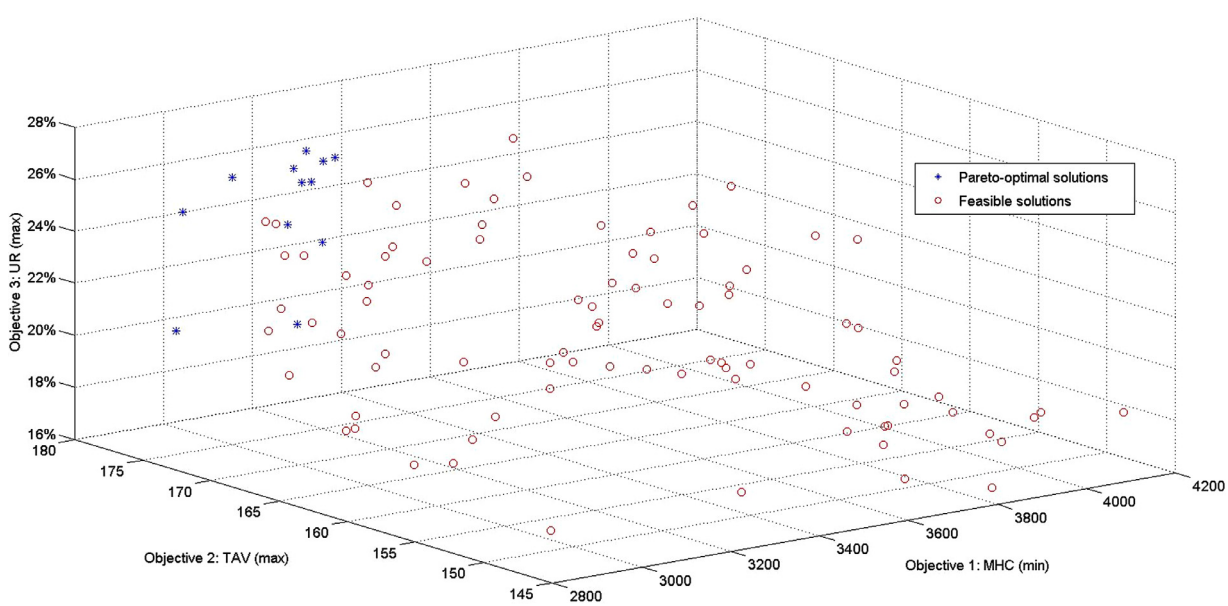
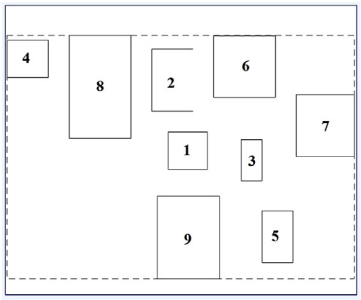
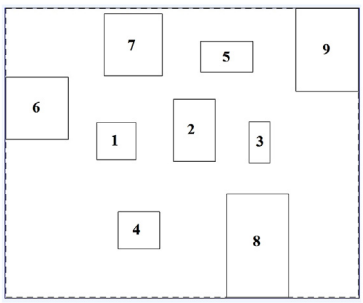


Fig. 7. The final solutions obtained by the MOPSO–OSD algorithm in a running for instance 1. (For interpretation of the references to color in this figure, the reader is referred to the web version of this article.)



(a) The optimal layout based on the preference of the objective 1



(b) The optimal layout based on the preference of the objective 2

Fig. 8. Two Pareto-optimal solutions with different preferences by the MOPSO–OSD algorithm for instance 2.

4.1. Computational results for a FLP with three objectives

A diesel engine factory needs to build a processing shop floor, and there are 12 facilities which need to be placed within the shop floor with given size in advance. The dimension of the shop floor is 12×10 m^{2}s. Taking into account the noise, dust, shake, heat quantity and safety among those facilities, every facility must keep certain space with other facilities. But we also need to set the facilities as compact as possible in order to avoid the waste of space in the shop floor. The dimension of each facility, minimum spacing requirement h_{ij} and material flows F_{ij} between any two different facilities a_i and a_j, and the matrix of adjacency requirement are presented in Tables 3, 4, 5, and 6, respectively. The unit cost matrix (the cost of a unit material handling per unit distance from facility a_i to a_j) is also shown in Table 7.

The computational results by the MOPSO–OSD algorithm for this instance are shown in Table 8. As shown in the table, all best objective values of the Pareto-optimal solutions gained by the pro-

Table 10
Dimensions of facilities.

Facility	1	2	3	4	5	6	7	8	9
Length (l_i)	1.9	3.0	2.0	2.0	2.5	3.0	3.0	5.0	3.0
Width (w_i)	1.8	2.0	1.0	1.8	1.5	3.0	2.8	3.0	4.0

Table 11
Matrix of a unit material handling cost C_{ij} per unit distance between facilities a_i and a_j.

Facility	1	2	3	4	5	6	7	8	9
1	0	5	2	1	8	6	3	1	2
2	5	0	3	2	2	4	3	4	2
3	2	3	0	2	3	2	4	3	3
4	1	2	2	0	2	3	2	7	3
5	8	2	3	2	0	2	2	3	2
6	6	4	2	3	2	0	5	6	2
7	3	3	4	2	2	5	0	4	1
8	1	4	3	7	3	6	4	0	3
9	2	2	3	3	2	2	1	3	0

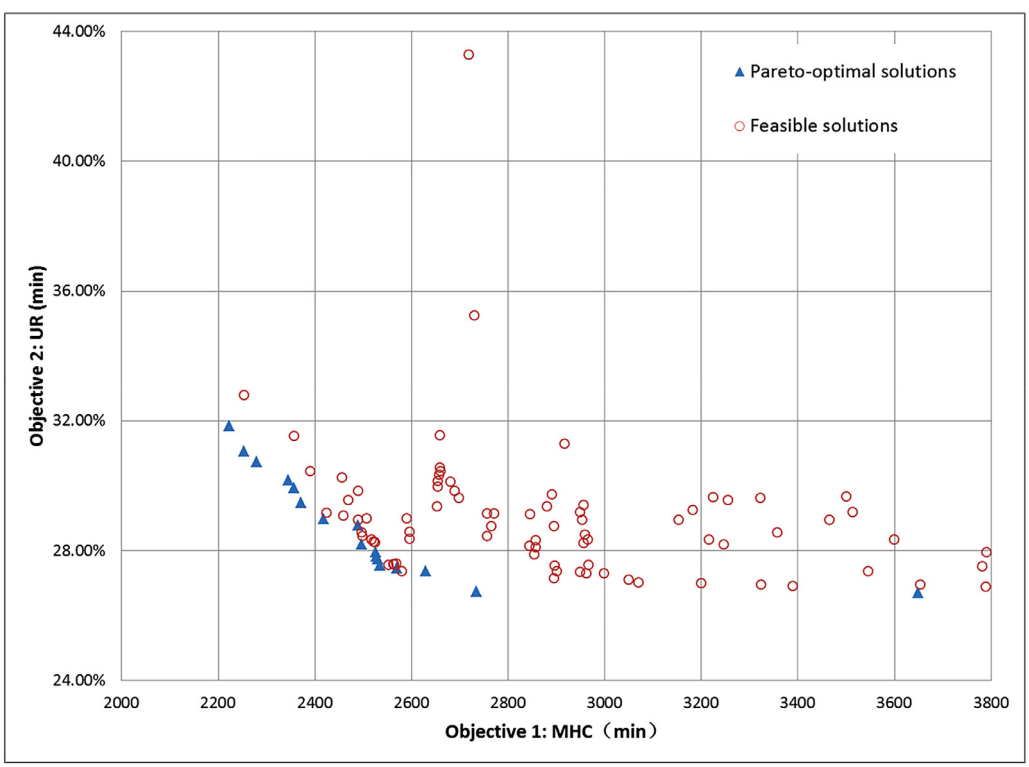


Fig. 9. The final solutions obtained by the MOPSO–OSD algorithm in a running for instance 2. (For interpretation of the references to color in this figure, the reader is referred to the web version of this article.)

Table 12
Material flows F_{ij} between facilities a_i and a_j.

Facility	1	2	3	4	5	6	7	8	9
1	0	1	2	1	1	0	1	2	4
2	1	0	2	3	0	2	1	2	0
3	2	2	0	0	2	3	1	2	1
4	1	3	0	0	1	1	1	3	1
5	1	0	2	1	0	1	2	1	2
6	0	2	3	1	1	0	1	1	3
7	1	1	1	1	2	1	0	0	1
8	2	2	2	3	1	1	0	0	1
9	4	0	1	1	2	3	1	1	0

Table 13
Comparison of the best computational results by three different algorithms for instance 2.

Objective	DECell (Zhang et al., 2013)	MOPSO–CD	MOPSO–OSD
MHC(min)	2898.8	2305.3	**2223.2**
UR(min)	31.54%	29.5%	**26.69%**

posed algorithm are uniformly better than those by the NSGA-II algorithm (Li et al., 2012). In order to test the effectiveness of the objective space division method in the MOPSO–OSD algorithm, the MOPSO algorithm with the crowding distance method (MOPSO–CD) instead of the objective space division method, is also run 10 times for this instance independently. As for the first objective, the material handling cost (MHC) (min), the best result obtained by the MOPSO–OSD algorithm is reduced by (2868.7 − 2856.54) / 2868.7 = 0.42% compared with that by the NSGA -II, but reduced by (2860.65 – 2856.54) / 2860.65 = 0.14% compared with that by the MOPSO–CD. As for the second objective, the total adjacency value (TAV) (max), the best result by the MOPSO–OSD algorithm is increased by (178 – 161.2) / 161.2 = 10.42% compared with that by the NSGA-II, and increased by (178 – 170.2) / 170.2 = 4.58% compared with that by the MOPSO–CD. As for the third objective, the utilization ratio (UR) of the shop floor (max), the best result by the MOPSO–OSD algorithm is increased by (27.04% – 19.3%) / 19.3% = 40.10% compared with that by the NSGA-II, and increased by (27.04% – 24.98%) / 24.98% = 8.25% compared with that by the MOPSO–CD. Obviously, the MOPSO–OSD is very competitive with the other approaches. The layouts with different preferences based on three objectives are shown in Fig. 3.

The statistics of the computational results for instance 1 are reported in Table 9. Table 9 lists the best and worst values of each objective, the average values, and the standard deviations of these values over 10 independent runs. The standard deviation is calculated by the following formula:

$$\sqrt{\frac{\sum_{i=1}^{T}\left(V_i-\bar{V}\right)^2}{T}} \tag{15}$$

where V_i denotes one objective value obtained by the MOPSO–OSD algorithm in the ith run; $\bar{V}$ denotes the average of these values and T is the number of runs. For the first objective, the standard deviation for all the values is 120.5. For the second and third objectives, the standard deviation is 3.31 and 0.02, respectively. Each standard deviation of the three objectives is small. This implies the MOPSO–OSD algorithm has stable performance in solving this problem.

Additionally, the evolution process of each objective and the Pareto-optimal set based on three objective functions has been determined. In this instance, the best set of all the non-dominated elements in the population (i.e. the Pareto-optimal set) is extracted from the results over ten independent runs, and the optimal values of three objectives at different stages of evolution process with the limit of 10,000 steps are shown in Figs. 4, 5, and 6, respectively.

From Figs. 4–6, we can see that the optimal values for the three objectives become better along with the evolution of the population. For objectives 1 and 2, after the step number of the iterations has increased to 6000, the changes of values tend to flatten and converge gradually. For objective 3, the objective values have a great fluctuation before 6000 steps, but after 8000 steps, the values tend to converge. This illustrates that during the process of searching the global optimal solution of the multi-objective problem, it takes other objectives' variation as the expense occasionally.

Fig. 7 shows all the final solutions obtained by the MOPSO–OSD algorithm in a running, where the Pareto-optimal solutions are denoted by the blue asterisks while the dominated solutions are represented by the red hollow circles. All the Pareto-optimal solutions are observed to form the Pareto front.

4.2. Computational results for a FLP with two objectives

The computational results for a FLP with two objectives are given in this subsection. This instance which includes 9 facilities is found in Zhang et al. (2013). The dimension of the shop floor is 17×14 m^2 s. Considering the noise between any two different facilities, the best use of given space and some activity space being set aside for the operators, the envelope rectangle area of all the facilities needs to be maximized within the specified scope. The safe distances of the horizontal and vertical directions between two different facilities are $d_{xmin} = 1$ m, and $d_{ymin} = 1$ m, respectively, where d_{xmin} represents the minimum space distance between any two different facilities in the horizontal direction while d_{ymin} in the vertical direction. The dimension of each facility, the unit cost matrix and material flows F_{ij} between any two different facilities a_i and a_j are given in Tables 10, 11 and 12, respectively. The objectives of the layout problem are to minimize material handling cost and to maximize the envelope rectangle area of all the facilities. Based on Eq. (3), there is need to convert the second objective to minimize utilization ratio (UR) of the shop floor.

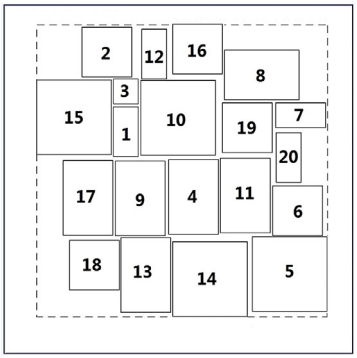
The computational results by the MOPSO–OSD algorithm for this instance are shown in Table 13. All best objective values of the Pareto-optimal solutions gained by the proposed algorithm are

Table 15
Dimension of the shop floor for a group of benchmark instances.

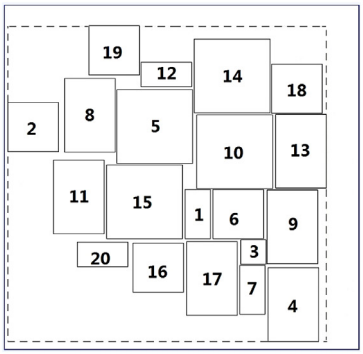
Instance	Number of facilities	Length of shop floor	Width of shop floor
Das08	8	40	40
Das10	10	50	50
Das12	12	50	50
SFLP8	8	12	12
SFLP11	11	15	15
SFLP20	20	14	14
Du62	62	140	150

Table 14
Statistics of results by the MOPSO–OSD algorithm for instance 2.

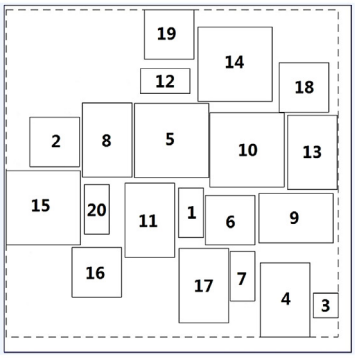
Objective	Best objective value	Worst objective value	Average objective value	Standard deviation
MHC(min)	2223.2	2799.2	2433.9	119.2
UR(min)	26.69%	37.9%	30.1%	0.03



(a) The optimal layout based on the preference of the objective 1

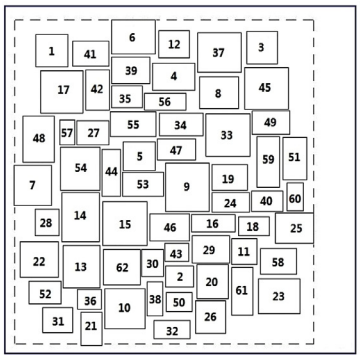


(b) The optimal layout based on the preference of the objective 2

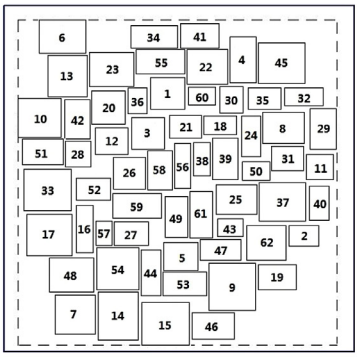


(c) The optimal layout based on the preference of the objective 3

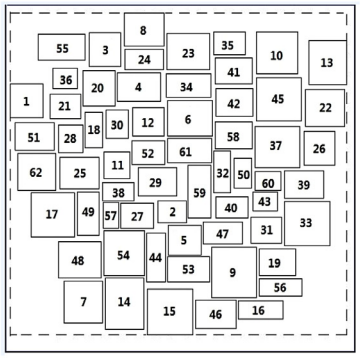
Fig. 10. Three Pareto-optimal solutions with different preferences by the MOPSO–OSD algorithm for SFLP20.



(a) The optimal layout based on the preference of the objective 1



(b) The optimal layout based on the preference of the objective 2



(c) The optimal layout based on the preference of the objective 3

Fig. 11. Three Pareto-optimal solutions with different preferences by the MOPSO–OSD algorithm for Du62.

uniformly better than those obtained by using the DECell algorithm (Zhang et al., 2013) and the MOPSO–CD algorithm. In fact, it decreased by (2898.8 – 2223.2)/2898.8 = 23.31% for material handling cost (MHC) (min) compared with that by the DECell algorithm and reduced by (2305.3 – 2223.2) / 2305.3 = 3.56% compared with that by the MOPSO–CD. As for the second objective, the utilization ratio (UR) of the shop floor (min), the best solution is decreased by (31.54% – 26.69%) / 31.54% = 15.38% compared with that by the DECell, and reduced by (29.5% – 26.69%)/29.5% = 9.52% compared with that by the MOPSO–CD. The optimal layouts with different preferences based on two objectives are shown in Fig. 8.

The statistics of the computational results for instance 2 are represented in Table 14. In this table, the best and worst values of each objective, the average values, and the standard deviations of these values over 10 independent runs are shown. For this instance, the small standard deviations of two objectives indicate that the MOPSO–OSD algorithm has as same stable performance as instance 1.

In addition, the solution set obtained by the MOPSO–OSD algorithm for instance 2 with two objectives is shown in Fig. 9. In the figure, we use the blue filled triangles to denote the Pareto-optimal solutions and the red hollow circles to represent the dominated solutions. Obviously, all the Pareto-optimal solutions clearly form the Pareto front.

4.3. Computational results for a group of benchmark instances with three objectives

To check further the effectiveness of the MOPSO–OSD algorithm, seven benchmark instances which are Das08, Das10, Das12

Table 16
Statistics of experimental results by three different algorithms for a group of benchmark instances.

		MHC(min)			TAV(max)			UR(max)		
Instance	Algorithm	Best	Worst	Average	Best	Worst	Average	Best	Worst	Average
Das08	MOPSO–OSD	**9420.69**	**11932.34**	**10115.63**	**166.56**	**149.39**	**153.81**	**89.35%**	**77.46%**	**80.13%**
	NSGA- II	9578.73	12384.77	10335.92	160.77	143.88	150.09	87.73%	72.19%	78.66%
	MOPSO–CD	9480.32	12138.86	10142.43	163.45	145.62	152.01	88.77%	73.21%	79.10%
Das10	MOPSO–OSD	**18403.88**	**21873.26**	**20398.41**	**283.51**	**239.19**	**259.77**	**87.68%**	**78.17%**	**80.53%**
	NSGA- II	18890.56	23192.23	21934.31	277.64	221.97	241.98	84.34%	73.23%	78.36%
	MOPSO–CD	18,810.24	22937.37	21123.95	280.11	232.13	253.82	85.78%	75.12%	80.02%
Das12	MOPSO–OSD	**39396.14**	**44202.13**	**42011.13**	**417.92**	**361.21**	**389.92**	**86.53%**	**76.49%**	**80.33%**
	NSGA- II	39923.12	45639.92	42934.25	409.15	349.33	375.66	83.45%	71.77%	76.98%
	MOPSO–CD	39734.99	45038.37	42534.16	415.23	355.12	384.92	84.88%	72.83%	79.56%
SFLP8	MOPSO–OSD	191.55	213.97	205.82	**235.22**	**201.24**	**217.89**	**86.42%**	**77.39%**	**80.12%**
	NSGA- II	198.45	227.38	217.88	219.26	182.26	199.52	85.11%	74.30%	78.22%
	MOPSO–CD	**188.75**	**210.34**	**202.74**	227.85	189.32	210.94	85.92%	74.98%	79.34%
SFLP11	MOPSO–OSD	**1286.11**	**1452.43**	**1335.64**	**282.13**	**236.92**	**253.81**	**86.68%**	**77.28%**	**80.35%**
	NSGA- II	1350.25	1529.23	1520.49	268.22	213.61	239.78	84.27%	74.13%	79.11%
	MOPSO–CD	1307.15	1503.40	1430.25	279.25	218.22	244.09	85.35%	76.65%	80.03%
SFLP20	MOPSO–OSD	**1206.65**	**1303.22**	**1264.23**	**468.36**	**379.92**	**410.39**	**80.11%**	**68.77%**	**72.94%**
	NSGA- II	1297.22	1598.30	1414.78	459.99	352.03	399.65	78.45%	64.37%	70.24%
	MOPSO–CD	1247.91	1513.22	1370.93	**468.36**	368.29	401.81	**80.11%**	65.82%	71.05%
Du62	MOPSO–OSD	**3965205.71**	**4219293.22**	**4210301.35**	**322.56**	**256.15**	**278.34**	**92.35%**	**84.88%**	**86.28%**
	NSGA- II	3997512.96	4353847.93	4220214.64	284.19	102.45	168.17	87.90%	80.15%	82.93%
	MOPSO–CD	3985604.25	4313493.83	4218854.65	306.66	159.43	207.13	89.49%	81.31%	84.12%

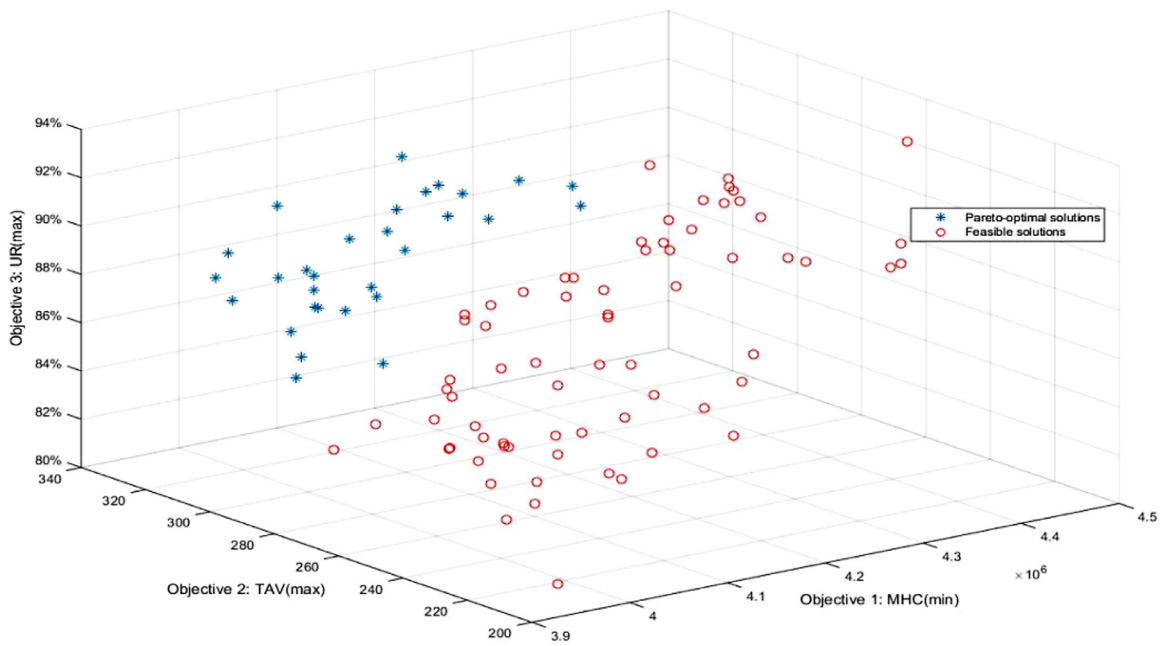


Fig. 12. The final solutions obtained by the MOPSO–OSD algorithm in a running for Du62. (For interpretation of the references to color in this figure, the reader is referred to the web version of this article.)

from Das (1993), SFLP8, SFLP11, SFLP20 from Asl and Wong (2017), and Du62 from Dunker, Radons, and Westkämper (2003) are tested for optimizing objectives (1)–(3). The digit in each instance name represents the number of facilities of this instance. The detailed data of these instances including the material flow, the width and height of each facility can be found in Das (1993), Asl and Wong (2017), and Dunker et al. (2003). It is worthwhile to mention that very few benchmark problems are available for the UA-FLP with multiple objectives, particularly in the case of TAV and UR. As a result, we have created matrix of adjacency requirement and the dimension of each shop floor (Table 15). The transportation cost C_{ij} of a unit material handling per unit distance between any two different facilities a_i and a_j is all set to 1. To justify the proposed algorithm, we implement also NSGA- II and MOPSO–CD for solving these instances. We compare the numerical results obtained by the MOPSO–OSD algorithm with those by the other two algorithms in Table 16. The best results are **bold** in the table.

From Table 16, it is evident that the best, the worst and average objective values obtained by the MOPSO–OSD algorithm for the MHC are better than those obtained by the other two approaches for all seven instances, except instance SFLP8. For instance SFLP8, the MOPSO–OSD algorithm performs slightly worse than the MOPSO–CD. In addition, from Table 16, one can see that the best, the worst and average objective values of TAV and UR obtained by the MOPSO–OSD algorithm are also desirable. It gives better or same results for all seven test instances in comparison to the NSGA-II and the MOPSO–CD. Especially for instance Du62, the MOPSO–OSD algorithm performs highly better than the other two algorithms. The numerical results show that the proposed algorithm has high effectiveness in dealing with bigger sized problems.

The optimal layouts with different preferences based on three objectives for SFLP20 and Du62 are shown in Fig. 10 and Fig. 11, respectively.

In addition, the solution set obtained by the MOPSO–OSD algorithm in a running for Du62 is shown in Fig. 12. In the figure, we use the blue asterisks to denote the Pareto-optimal solutions and the red hollow circles to represent the dominated solutions. As instances 1 and 2, all the Pareto-optimal solutions for Du62 form clearly the Pareto front.

In summary, the computational results of three groups of instances from the literature show that the proposed MOPSO–OSD algorithm successfully optimizes three objectives (MHC, TAV and UR) for the UA-FLP. The almost all properties of the proposed MOPSO–OSD algorithm are superior to other two algorithms in the literature. After the algorithm produces near-optimal layouts for the UA-FLPs, the decision makers can express their preferences on the basis of the knowledge of candidate solution set and choose the most demanding one among the generated layouts according to the requirements of the production targets or market circumstances.

5. Conclusions

Classical approaches to FLPs generally focus on the relative locations of equal-area facilities on a shop floor. However, in most applications and real-world scenarios, the FLPs have unequal-area facilities. It is also noticeable that the majority of the existing methods solves either the quantitative single objective (distance-based) or multi-objective FLPs using a weighted-sum method. In contrast, this research focuses on multi-objective unequal-area FLPs which are more realistic in real-world scenarios. The objectives of the problem include optimizing the MHC (quantitative aspect), the TAV and the UR (qualitative aspects) simultaneously. We develop a multi-objective particle swarm optimization (MOPSO) algorithm to solve the multi-objective UA-FLP. An objective space division (OSD) method in the MOPSO is proposed to improve the effects of the MOPSO. Comparison of the numerical results of the MOPSO–CD which replaces OSD by the crowding distance (CD) strategy and NSGA-II proves the feasibility of this improved method.

The proposed MOPSO–OSD algorithm is capable of finding a set of Pareto-optimal layouts that optimizes the MHC, the TAV and the UR throughout the entire evolutionary process. Thus, it provides a wide range of alternative choices, allowing decision makers to be more flexible and to make better decisions based on market circumstances. In further work, we will extend the idea of the proposed algorithm to researches and applications on the multi-objective unequal-area dynamic facility layout problem.

Acknowledgments

This work is supported by the National Natural Science Foundation of China (Grant Nos. 61373016, 61472147), and the Major Program of the National Social Science Foundation of China (Grant No. 16ZDA047).

References

Aiello, G., Enea, M., & Galante, G. (2006). A multi-objective approach to facility layout problem by genetic search algorithm and electre method. *Robotics and Computer-Integrated Manufacturing, 22*(5-6), 447–455.
Aiello, G., Scalia, G. L., & Enea, M. (2012). A multi-objective genetic algorithm for the facility layout problem based upon slicing structure encoding. *Expert Systems with Applications, 39*(12), 10352–10358.
Aiello, G., Scalia, G. L., & Enea, M. (2013). A non dominated ranking multi objective genetic algorithm and electre method for unequal area facility layout problems. *Expert Systems with Applications, 40*, 4812–4819.
Al-Hakim, L. (2000). On solving facility layout problems using genetic algorithms. *International Journal of Production Research, 38*(11), 2573–2582.
Asl, A. D., & Wong, K. Y. (2017). Solving unequal-area static and dynamic facility layout problems using modified particle swarm optimization. *Journal of Intelligent Manufacturing, 28*(6), 1317–1336.
Chang, M. S., & Ku, T. C. (2013). A slicing tree representation and QCP-model-based heuristic algorithm for the unequal-area block facility layout problem. *Mathematical Problems in Engineering, 2013*, 1–19.
Chwif, L., Barretto, M. R. P., & Moscato, L. A. (1998). A solution to the facility layout problem using simulated annealing. *Computers in Industry, 36*(1–2), 125–132.
Das, S,K. (1993). A facility layout method for flexible manufacturing systems. *International Journal of Production Research, 31*(2), 279–297.
Diego, M. J. A., Santamarina, S. M. C., Alcaide, M. J., & Cloquell, B. V. A. (2009). Solving facility layout problems with strict geometric constraints using a two phase genetic algorithm. *International Journal of Production Research, 6*(15), 1679–1693.
Drira, A., Pierreval, H., & Hajri-Gabouj, S. (2007). Facility layout problems: a survey. *Annual Reviews in Control, 31*(2), 255–267.
Dunker, T., Radons, G., & Westkämper, E. (2003). A coevolutionary algorithm for a facility layout problem. *International Journal of Production Research, 41*(15), 3479–3500.
Engelbrecht, A. P. (2007). *Computational intelligence: An introduction*. Chichester: Wiley.
García-Hernández, L., Pérez-Ortiz, M., Araúzo-Azofra, A., Salas-Morera, L., & Hervás–Martínez, C. (2014). An evolutionary neural system for incorporating expert knowledge into the UA-FLP☆. *Neurocomputing, 135*(13), 69–78.
Gómez, A., Fernández, Q. I., Garcıá, D. D. L. F., & Garcıá, P. J. (2003). Using genetic algorithms to resolve layout problems in facilities where there are aisles. *International Journal of Production Economics, 84*(3), 271–282.
Gonçalves, J. F., & Resende, M. G. (2015). A biased random-key genetic algorithm for the unequal area facility layout problem. *European Journal of Operational Research, 246*(1), 86–107.
Guan, J., & Lin, G. (2016). Hybridizing variable neighborhood search with ant colony optimization for solving the single row facility layout problem. *European Journal of Operational Research, 248*(3), 899–909.
Islier, A. A. (1998). A genetic algorithm approach for multiple criteria facility layout design. *International Journal of Production Research, 36*(6), 1549–1569.
Jolai, F., Tavakkoli-Moghaddam, R., & Taghipour, M. (2012). A multi-objective particle swarm optimization algorithm for unequal sized dynamic facility layout problem with pickup/drop-off locations. *International Journal of Production Research, 50*(15), 4279–4293.
Kang, S., & Chae, J. (2017). Harmony search for the layout design of an unequal area facility. *Expert Systems with Applications, 79*, 268–281.
Kennedy, J., & Eberhart, R. (1995). Particle swarm optimization. In *Proceedings of the IEEE international conference on neural networks: 4* (pp. 1942–1948). Perth, WA, Australia IEEE.
Khilwani, N., Shankar, R., & Tiwari, M. K. (2008). Facility layout problem: an approach based on a group decision-making system and psychoclonal algorithm. *International Journal of Production Research, 46*(4), 895–927.
Komarudin, & Wong, K. Y. (2010). Applying ant system for solving unequal area facility layout problems. *European Journal of Operational Research, 202*(3), 730–746.
Kulturel-Konak, S., & Konak, A. (2010). Unequal area flexible bay facility layout using ant colony optimization. *International Journal of Production Research, 49*(7), 1877–1902.
Kulturel-Konak, S., & Konak, A. (2015). Large-scale hybrid simulated annealing algorithm for cyclic facility layout problems. *Engineering Optimization, 47*(7), 963–978.
Lee, K. Y., Han, S. N., & Roh, M. I. (2003). An improved genetic algorithm for facility layout problems having inner structure walls and passages. *Computers & Operations Research, 30*(1), 117–138.
Li, A. P., Yan, Z. Y., Xie, N., & Huang, J. Z. (2012). A research of multi-objective facility layout based on NSGA-II. *Machine Design and Research, 28*(6), 90–95.
Liu, J. F., Jiang, Y. C., Li, G., Xue, Y., Liu, Z. X., & Zhang, Z. (2015). Heuristic-based energy landscape paving for the circular packing problem with performance constraints of equilibrium. *Physica A: Statistical Mechanics and its Applications, 431*, 166–174.
Liu, J. F., & Li, G. (2010). Basin filling algorithm for the circular packing problem with equilibrium behavioral constraints. *Science China Information Science, 53*(5), 885–895.
Liu, J. F., Wang, D. W., He, K., & Xue, Y. (2017). Combining Wang–Landau sampling algorithm and heuristics for solving the unequal-area dynamic facility layout problem. *European Journal of Operational Research, 262*(3), 1052–1063.
Liu, J. F., Xue, S. J., Liu, Z. X., & Xu, D. H. (2009). An improved energy landscape paving algorithm for the problem of packing circles into a larger containing circle. *Computers & Industrial Engineering, 57*(3), 1144–1149.
Mckendall, A. R., & Hakobyan, A. (2010). Heuristic for the dynamic facility layout problem with unequal-area departments. *European Journal of Operational Research, 201*(1), 171–182.
Muther, R. (1973). *Systematic layout planning*. Boston: Cahners Books.
Nordin, N. N., Zainuddin, Z. M., Salim, S., & Ponnusamy, R. R. (2009). Mathematical modeling and hybrid heuristic for unequal size facility layout problem. *Journal of Fundamental Sciences, 5*(1), 742–756 79-87. 256(3).
Paes, F. G., Pessoa, A. A., & Vidal, T. (2017). A hybrid genetic algorithm with decomposition phases for the unequal area facility layout problem. *European Journal of Operational Research, 256*(3), 742–756.
Palomo-Romero, J. M., Salas-Morera, L., & García-Hernández, L. (2017). An island model genetic algorithm for unequal area facility layout problems. *Expert Systems with Applications, 68*, 151–162.

Saraswat, A., Venkatadri, U., & Castillo, I. (2015). A framework for multi-objective facility layout design. *Computers & Industrial Engineering, 90*, 167–176.

Tompkins, J. A., White, J. A., Bozer, Y. A., & Tanchoco, J. M. A. (2010). *Facilities planning* (4th ed.). New York: Wiley.

Ulutas, B. H., & Kulturel-Konak, S. (2012). An artificial immune system based algorithm to solve unequal area facility layout problem. *Expert Systems with Applications, 39*, 5384–5395.

Vitayasak, S., Pongcharoen, P., & Hicks, C. (2017). A tool for solving stochastic dynamic facility layout problems with stochastic demand using either a genetic algorithm or modified backtracking search algorithm. *International Journal of Production Economics, 190*, 146–157.

Wong, K. Y., & Komarudin (2010). Solving facility layout problems using flexible bay structure representation and ant system algorithm. *Expert Systems with Applications, 37*(7), 5523–5527.

Wu, Y., & Appleton, E. (2002). The optimization of block layout and aisle structure by a genetic algorithm. *Computers & Industrial Engineering, 41*(4), 371–387.

Zhang, Y., Lu, C., Zhang, H., & Fang, Z. F. (2013). Workshop layout optimization based on differential cellular multi-objective genetic algorithm. *Computer Integrated Manufacturing Systems, 19*(4), 727–734.

Zhou, G. G., Ye, M. J., Cao, Z. Y., & Ye, F. (2006). A genetic algorithm approach on a facility layout design problem with aisles. In D. S. Huang, K. Li, & G. W. Irwin (Eds.). In *Proceedings of the International conference on intelligent computing: 4114* (pp. 1008–1013). Berlin, Heidelberg: Springer-Verlag.